

# Solución de la ecuación de calor

Método de diferencias finitas y solución analítica por separación de variables

M. en C. Jeoshua Alejandro Licea Garcia  
ICN-UNAM

## Resumen

Este mini proyecto estudia la ecuación unidimensional de difusión térmica mediante dos enfoques: una solución numérica basada en diferencias finitas explícitas y una solución analítica obtenida mediante separación de variables. Se utilizan propiedades físicas de un material conductor para calcular la difusividad térmica y el paso temporal. Finalmente, se comparan ambas soluciones para verificar el comportamiento del método numérico.

## Índice

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                        | <b>2</b>  |
| <b>2. Parámetros físicos y discretización</b>                                 | <b>2</b>  |
| <b>3. Implementación computacional</b>                                        | <b>3</b>  |
| <b>4. Obtención del método numérico mediante series de Taylor</b>             | <b>3</b>  |
| 4.1. Aproximación de la derivada temporal mediante series de Taylor . . . . . | 4         |
| 4.2. Aproximación de la segunda derivada espacial . . . . .                   | 4         |
| 4.2.1. Expansión hacia adelante . . . . .                                     | 5         |
| 4.2.2. Expansión hacia atrás . . . . .                                        | 5         |
| 4.3. Suma de las expansiones . . . . .                                        | 5         |
| 4.4. Sustitución en la ecuación de calor . . . . .                            | 6         |
| 4.5. Relación con el código . . . . .                                         | 6         |
| <b>5. Condiciones iniciales y de frontera</b>                                 | <b>7</b>  |
| <b>6. Avance temporal y almacenamiento</b>                                    | <b>7</b>  |
| <b>7. Solución analítica mediante separación de variables</b>                 | <b>7</b>  |
| <b>8. Implementación de la solución analítica</b>                             | <b>9</b>  |
| <b>9. Gráficas obtenidas del código</b>                                       | <b>9</b>  |
| <b>10.Comparación: diferencias finitas vs. solución analítica</b>             | <b>13</b> |
| <b>11.Visualización de todos los estados</b>                                  | <b>14</b> |
| <b>12.Conclusiones</b>                                                        | <b>14</b> |

## 1. Introducción

La ecuación de calor describe la evolución temporal de la temperatura en un medio donde existe conducción térmica. Para una barra unidimensional de longitud  $L$ , con propiedades constantes y sin fuentes internas, el modelo es

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad (1)$$

donde  $T(x, t)$  es la temperatura y  $\alpha$  es la difusividad térmica, definida como

$$\alpha = \frac{k}{\rho c}. \quad (2)$$

En esta expresión,  $k$  representa la conductividad térmica del material,  $\rho$  su densidad y  $c$  su calor específico. Para el material considerado en este proyecto se utilizan los siguientes valores:

$$k = 210 \text{ W/(m K)}, \quad (3)$$

$$\rho = 2700 \text{ kg/m}^3, \quad (4)$$

$$c = 900 \text{ J/(kg K)}. \quad (5)$$

Por lo tanto, la difusividad térmica es

$$\alpha = \frac{210 \text{ W/(m K)}}{(2700 \text{ kg/m}^3)(900 \text{ J/(kg K)})} \approx 8.642 \times 10^{-5} \text{ m}^2/\text{s}. \quad (6)$$

La difusividad térmica  $\alpha$  caracteriza la rapidez con la que una perturbación de temperatura se propaga a través del material. Sus unidades son  $\text{m}^2/\text{s}$ .

En este proyecto se considera una barra en el intervalo  $0 \leq x \leq 1$  con temperatura inicialmente elevada en su interior y extremos mantenidos a temperatura cero. El problema permite observar cómo el calor se difunde desde la región interior hacia las fronteras (típica intuición de lo caliente a lo frío).

El código fue construido utilizando arreglos de NumPy y una implementación explícita del método de diferencias finitas. Además, se obtiene una solución analítica para disponer de una referencia independiente con la cual evaluar el método numérico.

## 2. Parámetros físicos y discretización

Los parámetros utilizados en el programa son

| Parámetro | Valor | Descripción                 |
|-----------|-------|-----------------------------|
| $N_x$     | 100   | Número de celdas espaciales |
| $x_l$     | 0     | Límite izquierdo            |
| $x_r$     | 1     | Límite derecho              |
| $c$       | 900   | Calor específico            |
| $\rho$    | 2700  | Densidad                    |
| $k$       | 210   | Conductividad térmica       |
| CFL(C)    | 0.01  | Parámetro de estabilidad    |

Cuadro 1: Parámetros principales del modelo.

La separación espacial se calcula mediante

$$\Delta x = \frac{x_r - x_l}{N_x - 1}. \quad (7)$$

El **CFL (Courant–Friedrichs–Lewy)** es un número adimensional que relaciona el paso temporal con el espacial para controlar la estabilidad del método numérico.

$$C = \frac{\alpha \Delta t}{\Delta x^2}. \quad (8)$$

Para este método se requiere  $CFL \leq 1/2$  y en el proyecto se utiliza  $CFL = 0.01$ . Que nos arroja un  $\Delta t$  dado por:

```
1 def timestep(): #PASO DE TIEMPO
2     return cfl * dx**2 * rho * c / k
```

### 3. Implementación computacional

Las bibliotecas principales son NumPy para el manejo de arreglos y Matplotlib para la visualización.

```
1 #Solucion ecuacion de calor por modulos M en C. JEOSHUA LICEA GARCIA ICN-UNAM
2 #contacto jeoshua.licea@correo.nucleares.unam.mx
3 import numpy as np
4 import matplotlib.pyplot as plt
5 #Coeficientes fisicos reales
6 Nx = 100
7 xr = 1
8 xl = 0
9 dx = (xr - xl) / (Nx-1)
10 c = 900
11 rho = 2700
12 k = 210
13 t = 0
14 it = 0
15 dtout = 5
16 tfin = 60
17 cfl = 0.01
18
19 x = np.arange(xl-dx, xr+2*dx, dx)
20
21 T = [0.0 for _ in range(Nx+2)]
22 TP = [0.0 for _ in range(Nx+2)] #con las fantasmas
```

El arreglo espacial contiene  $N_x + 2$  puntos porque se incluyen los dos puntos de frontera además de los puntos interiores.

### 4. Obtención del método numérico mediante series de Taylor

En esta sección desarrollo paso a paso el esquema numérico utilizado para resolver la ecuación de calor. Las aproximaciones de diferencias finitas se obtienen a partir de las expansiones en series de Taylor alrededor de un punto de la malla.

Se parte de la ecuación de calor unidimensional

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad (9)$$

Para obtener una solución numérica, se discretizan tanto el espacio como el tiempo. Definimos los puntos de la malla como

$$x_i = x_0 + i\Delta x, \quad (10)$$

y los instantes de tiempo como

$$t^n = t_0 + n\Delta t. \quad (11)$$

Por simplicidad, utilizaremos la notación

$$T_i^n = T(x_i, t^n). \quad (12)$$

El objetivo es obtener una expresión que permita calcular  $T_i^{n+1}$  a partir de los valores conocidos en el tiempo anterior  $n$ .

#### 4.1. Aproximación de la derivada temporal mediante series de Taylor

Sea la función  $T(x_i, t)$  y expandámosla alrededor del instante  $t^n$ . La serie de Taylor hacia adelante es

$$T(x_i, t^n + \Delta t) = T(x_i, t^n) + \Delta t \left. \frac{\partial T}{\partial t} \right|_i^n + \frac{\Delta t^2}{2!} \left. \frac{\partial^2 T}{\partial t^2} \right|_i^n + \frac{\Delta t^3}{3!} \left. \frac{\partial^3 T}{\partial t^3} \right|_i^n + \dots \quad (13)$$

Utilizando la notación discreta,

$$T_i^{n+1} = T_i^n + \Delta t \left. \frac{\partial T}{\partial t} \right|_i^n + \frac{\Delta t^2}{2} \left. \frac{\partial^2 T}{\partial t^2} \right|_i^n + \mathcal{O}(\Delta t^3). \quad (14)$$

Al despejar la primera derivada temporal:

$$T_i^{n+1} - T_i^n = \Delta t \left. \frac{\partial T}{\partial t} \right|_i^n + \frac{\Delta t^2}{2} \left. \frac{\partial^2 T}{\partial t^2} \right|_i^n + \mathcal{O}(\Delta t^3). \quad (15)$$

Dividiendo entre  $\Delta t$ ,

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \left. \frac{\partial T}{\partial t} \right|_i^n + \frac{\Delta t}{2} \left. \frac{\partial^2 T}{\partial t^2} \right|_i^n + \mathcal{O}(\Delta t^2). \quad (16)$$

Por lo tanto,

$$\left. \frac{\partial T}{\partial t} \right|_i^n = \frac{T_i^{n+1} - T_i^n}{\Delta t} - \frac{\Delta t}{2} \left. \frac{\partial^2 T}{\partial t^2} \right|_i^n + \mathcal{O}(\Delta t^2). \quad (17)$$

Si se conserva únicamente el primer término de la aproximación, se obtiene

$$\boxed{\left. \frac{\partial T}{\partial t} \right|_i^n \approx \frac{T_i^{n+1} - T_i^n}{\Delta t}} \quad (18)$$

Esta expresión corresponde a una **diferencia finita hacia adelante**. El término que se desprecia es proporcional a  $\Delta t$ , por lo que el error de truncamiento local de esta aproximación es de primer orden:

$$\boxed{E_t = \mathcal{O}(\Delta t)} \quad (19)$$

#### 4.2. Aproximación de la segunda derivada espacial

Para obtener una aproximación de la segunda derivada espacial, se desarrollan dos series de Taylor alrededor del punto  $x_i$ . Una se desplaza hacia la derecha y la otra hacia la izquierda.

#### 4.2.1. Expansión hacia adelante

La expansión de Taylor de  $T(x_i + \Delta x, t^n)$  es

$$T(x_i + \Delta x, t^n) = T_i^n + \Delta x \frac{\partial T}{\partial x} \Big|_i^n + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} \Big|_i^n + \frac{\Delta x^3}{6} \frac{\partial^3 T}{\partial x^3} \Big|_i^n + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} \Big|_i^n + \dots \quad (20)$$

En notación discreta,

$$T_{i+1}^n = T_i^n + \Delta x T_x + \frac{\Delta x^2}{2} T_{xx} + \frac{\Delta x^3}{6} T_{xxx} + \frac{\Delta x^4}{24} T_{xxxx} + \dots \quad (21)$$

#### 4.2.2. Expansión hacia atrás

De manera análoga, para  $x_i - \Delta x$ ,

$$T(x_i - \Delta x, t^n) = T_i^n - \Delta x \frac{\partial T}{\partial x} \Big|_i^n + \frac{\Delta x^2}{2} \frac{\partial^2 T}{\partial x^2} \Big|_i^n - \frac{\Delta x^3}{6} \frac{\partial^3 T}{\partial x^3} \Big|_i^n + \frac{\Delta x^4}{24} \frac{\partial^4 T}{\partial x^4} \Big|_i^n + \dots \quad (22)$$

Por lo tanto,

$$T_{i-1}^n = T_i^n - \Delta x T_x + \frac{\Delta x^2}{2} T_{xx} - \frac{\Delta x^3}{6} T_{xxx} + \frac{\Delta x^4}{24} T_{xxxx} + \dots \quad (23)$$

#### 4.3. Suma de las expansiones

Ahora sumamos las dos expresiones:

$$\begin{aligned} T_{i+1}^n + T_{i-1}^n &= 2T_i^n + \Delta x T_x - \Delta x T_x \\ &\quad + \frac{\Delta x^2}{2} T_{xx} + \frac{\Delta x^2}{2} T_{xx} \\ &\quad + \frac{\Delta x^3}{6} T_{xxx} - \frac{\Delta x^3}{6} T_{xxx} \\ &\quad + \frac{\Delta x^4}{24} T_{xxxx} + \frac{\Delta x^4}{24} T_{xxxx} + \dots \end{aligned} \quad (24)$$

Los términos de derivadas impares se cancelan. Por lo tanto,

$$T_{i+1}^n + T_{i-1}^n = 2T_i^n + \Delta x^2 T_{xx} + \frac{\Delta x^4}{12} T_{xxxx} + \mathcal{O}(\Delta x^6). \quad (25)$$

Restando  $2T_i^n$ ,

$$T_{i+1}^n - 2T_i^n + T_{i-1}^n = \Delta x^2 T_{xx} + \frac{\Delta x^4}{12} T_{xxxx} + \mathcal{O}(\Delta x^6). \quad (26)$$

Dividiendo entre  $\Delta x^2$ ,

$$\frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2} = T_{xx} + \frac{\Delta x^2}{12} T_{xxxx} + \mathcal{O}(\Delta x^4). \quad (27)$$

Finalmente, despejamos la segunda derivada:

$$\frac{\partial^2 T}{\partial x^2} \Big|_i^n = \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2} - \frac{\Delta x^2}{12} T_{xxxx} + \mathcal{O}(\Delta x^4). \quad (28)$$

Si despreciamos los términos de orden superior, obtenemos la aproximación de diferencia centrada:

$$\left. \frac{\partial^2 T}{\partial x^2} \right|_i^n \approx \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2} \quad (29)$$

En este caso, el error de truncamiento espacial es de segundo orden:

$$E_x = \mathcal{O}(\Delta x^2) \quad (30)$$

#### 4.4. Sustitución en la ecuación de calor

Ya tenemos las dos aproximaciones necesarias:

$$\frac{\partial T}{\partial t} \approx \frac{T_i^{n+1} - T_i^n}{\Delta t}, \quad (31)$$

y

$$\frac{\partial^2 T}{\partial x^2} \approx \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2}. \quad (32)$$

Sustituyendo ambas expresiones en la ecuación de calor,

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha \frac{T_{i+1}^n - 2T_i^n + T_{i-1}^n}{\Delta x^2}. \quad (33)$$

Ahora realizamos el despeje paso a paso. Multiplicamos ambos lados por  $\Delta t$ :

$$T_i^{n+1} - T_i^n = \frac{\alpha \Delta t}{\Delta x^2} (T_{i+1}^n - 2T_i^n + T_{i-1}^n). \quad (34)$$

Sumamos  $T_i^n$  a ambos lados:

$$T_i^{n+1} = T_i^n + \frac{\alpha \Delta t}{\Delta x^2} (T_{i+1}^n - 2T_i^n + T_{i-1}^n). \quad (35)$$

Entonces el esquema numérico queda finalmente como

$$T_i^{n+1} = T_i^n + C (T_{i+1}^n - 2T_i^n + T_{i-1}^n) \quad (36)$$

Esta expresión permite avanzar la solución un paso en el tiempo utilizando únicamente los valores conocidos del instante anterior.

#### 4.5. Relación con el código

La expresión matemática obtenida se implementa directamente en la función `difusion()`:

```
1 def difusion(T, TP, cfl): #METODO NUMERICO DE DIFERENCIA FINITA
2     for i in range(1, Nx + 1): #puras celdas reales
3         TP[i] = T[i] + cfl * (T[i + 1] + T[i - 1] - 2 * T[i])
4     return TP
```

La correspondencia entre la ecuación y el código es

| Expresión matemática | Código |
|----------------------|--------|
| $T_i^{n+1}$          | TP[i]  |
| $T_i^n$              | T[i]   |
| $T_{i+1}^n$          | T[i+1] |
| $T_{i-1}^n$          | T[i-1] |
| $C$                  | cfl    |
| $\Delta x$           | dx     |
| $\Delta t$           | dt     |

Por lo tanto, la línea

```
1 TP[i] = T[i] + cfl * (T[i + 1] + T[i - 1] - 2 * T[i])
```

es la implementación directa del esquema explícito obtenido mediante las series de Taylor.

## 5. Condiciones iniciales y de frontera

La función de condición inicial del código es

```
1 def inittemperature(T): #CONDICION INICIAL
2     for i in range(1,Nx+1):
3         T[i] = 100
```

y las fronteras se fijan mediante

```
1 def boundary(T): #FRONTERAS sobre las fantasmas
2     T[0] = 0
3     T[Nx + 1] = 0
```

Matemáticamente, el problema físico que se desea representar es

$$T(x, 0) = 100, \quad 0 < x < 1, \quad (37)$$

con

$$T(0, t) = T(1, t) = 0. \quad (38)$$

## 6. Avance temporal y almacenamiento

La función `step()` ejecuta la simulación completa.

```
1 def step(T, TP, cfl, t, dt, it, tfin, dtout): #PASO Y ESCRITURA A DISCO
2     CU = [T.copy()]
3     next_output = dtout
4
5     while t < tfin:
6         TP = difusion(T, TP, cfl)
7         T = TP.copy()
8         boundary(T)
9         t += dt
10        it += 1
11        if t >= next_output:
12            CU.append(T.copy())
13            next_output += dtout
14
15    print("Simulacion terminada con exito")
16    return T, TP, t, it, CU
```

El uso de `copy()` es importante porque permite guardar un snap (*snapshot*) independiente del estado actual del arreglo.

## 7. Solución analítica mediante separación de variables

Consideremos el problema

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad 0 < x < L, \quad (39)$$

con

$$T(0, t) = T(L, t) = 0 \quad (40)$$

y condición inicial

$$T(x, 0) = T_0. \quad (41)$$

Buscamos una solución separable

$$T(x, t) = X(x)\Theta(t). \quad (42)$$

Sustituyendo,

$$X(x) \frac{d\Theta}{dt} = \alpha \Theta(t) \frac{d^2 X}{dx^2}. \quad (43)$$

Dividiendo entre  $\alpha X \Theta$ ,

$$\frac{1}{\alpha \Theta} \frac{d\Theta}{dt} = \frac{1}{X} \frac{d^2 X}{dx^2} = -\lambda. \quad (44)$$

Se obtienen dos ecuaciones ordinarias:

$$\frac{d\Theta}{dt} + \alpha \lambda \Theta = 0, \quad (45)$$

y

$$\frac{d^2 X}{dx^2} + \lambda X = 0. \quad (46)$$

Con solución para  $\Theta(t)$  de tipo separable

$$\ln(\Theta(t)) = -\alpha \lambda t, \quad \Theta(t) = e^{-\alpha \lambda t} \quad (47)$$

Mientras que para  $X(x)$ , se tiene el caso

$$X(x) = A_n \cos(\sqrt{\lambda} x) + B_n \sin(\sqrt{\lambda} x) \quad (48)$$

Las condiciones  $X(0) = X(L) = 0$  producen los eigenvalores, como sigue:

$$X(0) = A_n \cos(0) + B_n \sin(0) = 0, \quad A_n = 0 \quad (49)$$

es decir

$$X(x) = B_n \sin(\sqrt{\lambda} x) \quad (50)$$

luego

$$X(L) = B_n \sin(\sqrt{\lambda} L) = 0 \quad (51)$$

produce la discretización

$$\lambda_n = \left(\frac{n\pi}{L}\right)^2 \quad n \in \mathbf{Z} \quad (52)$$

y las autofunciones

$$X_n(x) = B_n \sin\left(\frac{n\pi x}{L}\right). \quad (53)$$

La parte temporal es

$$\Theta_n(t) = \exp\left[-\alpha\left(\frac{n\pi}{L}\right)^2 t\right]. \quad (54)$$

Por superposición,

$$T(x, t) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right) \exp\left[-\alpha\left(\frac{n\pi}{L}\right)^2 t\right]. \quad (55)$$

cuando  $t = 0$ , sucede

$$T(x, 0) = \sum_{n=1}^{\infty} B_n \sin\left(\frac{n\pi x}{L}\right). \quad (56)$$

Los coeficientes se obtienen mediante la serie de Fourier seno:

$$B_n = \frac{2}{L} \int_0^L T_0 \sin\left(\frac{n\pi x}{L}\right) dx. \quad (57)$$

Para  $T(x, 0) = 100$ ,

$$B_n = \frac{200}{n\pi} (1 - (-1)^n). \quad (58)$$

Así, los términos pares desaparecen y sólo permanecen los impares:

$$T(x, t) = \frac{400}{\pi} \sum_{\substack{n=1 \\ n \text{ impar}}}^{\infty} \frac{1}{n} \sin\left(\frac{n\pi x}{L}\right) e^{-\alpha(n\pi/L)^2 t} \quad (59)$$

Para este proyecto  $L = 1$ , por lo que

$$T(x, t) = \frac{400}{\pi} \sum_{\substack{n=1 \\ n \text{ impar}}}^{\infty} \frac{\sin(n\pi x)}{n} e^{-\alpha n^2 \pi^2 t}. \quad (60)$$

## 8. Implementación de la solución analítica

La serie infinita se aproxima computacionalmente utilizando un número finito de términos:

```

1 modes = np.arange(1, 1000, 2)
2 alpha = k / (c * rho)
3 T_analytic = np.zeros_like(x)
4
5 for m in modes:
6     T_analytic += (
7         (1.0 / m)
8         * np.sin(m * np.pi * x / L)
9         * np.exp(-alpha * (m * np.pi / L) ** 2 * tfin)
10    )
11
12 T_analytic *= 400.0 / np.pi
13 plt.figure(figsize=(8, 5))
14 plt.plot(x, T_analytic, color="firebrick", lw=2, label=f"Analitico (t = {tfin}s)
15 ")
16 plt.title(f"Perfil de Temperatura T(x, t) a t = {tfin} s")
17 plt.xlabel("Posicin x (m)")
18 plt.ylabel("Temperatura T (C)")
19 plt.xlim(0, L)
20 plt.grid(True, linestyle="--", alpha=0.6)
21 plt.legend()
22 plt.show()

```

El truncamiento en 1000 términos es más que suficiente para este instante de tiempo, ya que los términos de alta frecuencia están fuertemente amortiguados por el factor exponencial.

## 9. Gráficas obtenidas del código

Las siguientes figura corresponden a la salida del programa proporcionada para cada  $t = 0$  s hasta  $t = 60$  s.

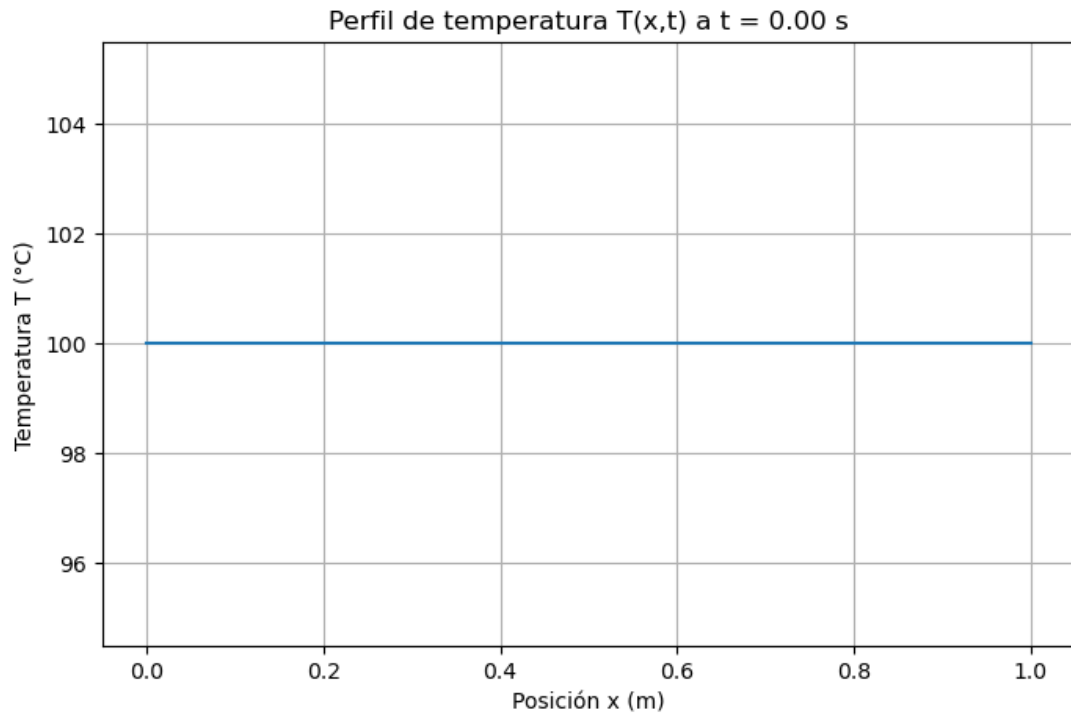


Figura 1: Perfil de temperatura obtenido mediante el código numérico a  $t = 0$  s (Condición inicial).

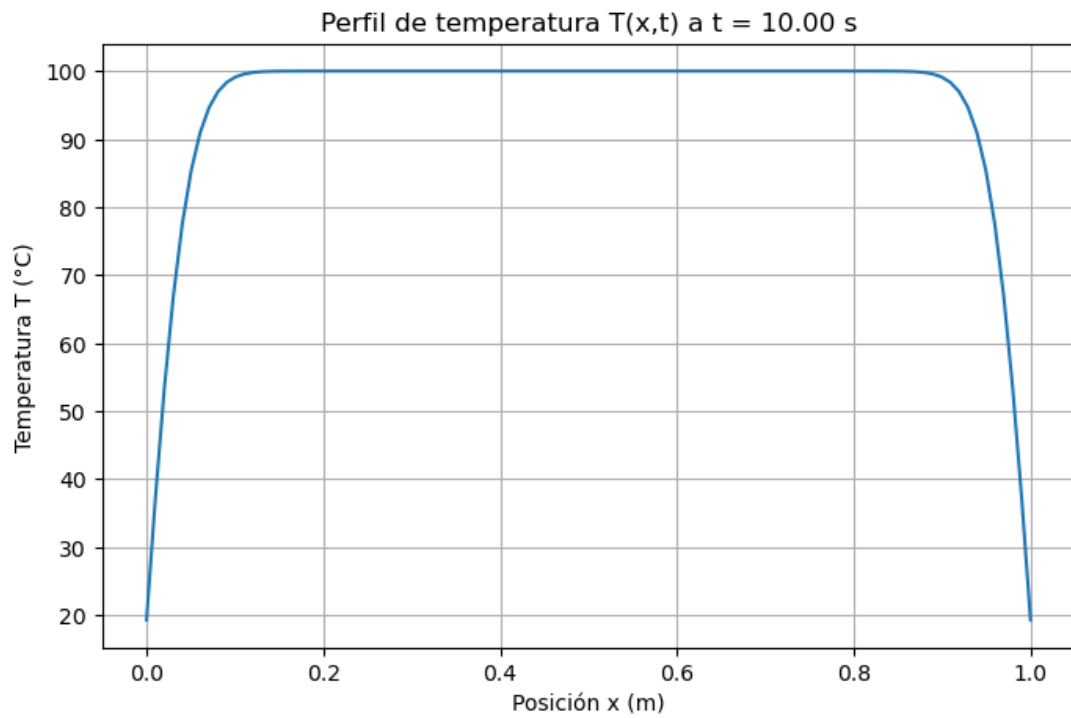


Figura 2: Perfil de temperatura obtenido mediante el código numérico a  $t = 10$  s.

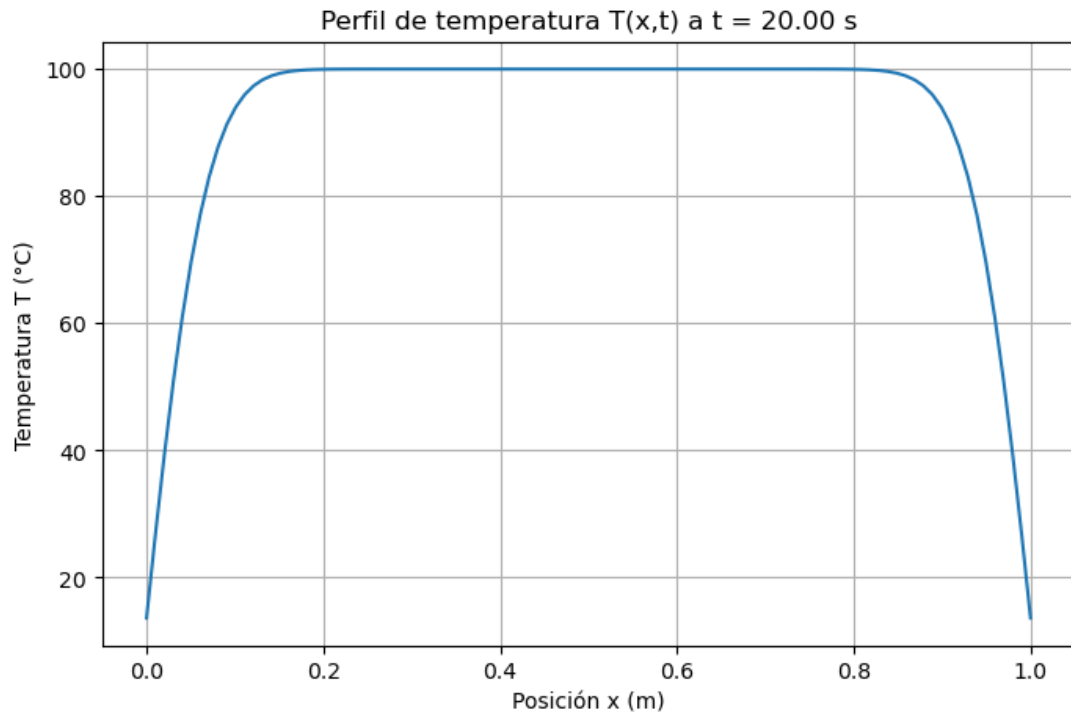


Figura 3: Perfil de temperatura obtenido mediante el código numérico a  $t = 20$  s.

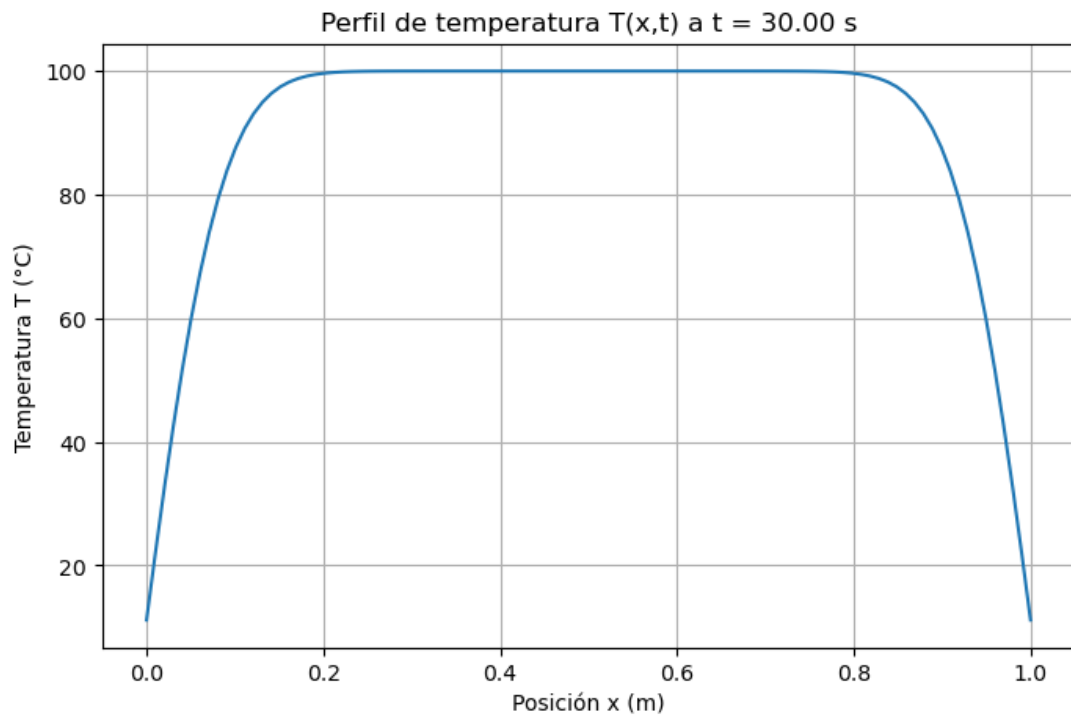


Figura 4: Perfil de temperatura obtenido mediante el código numérico a  $t = 30$  s.

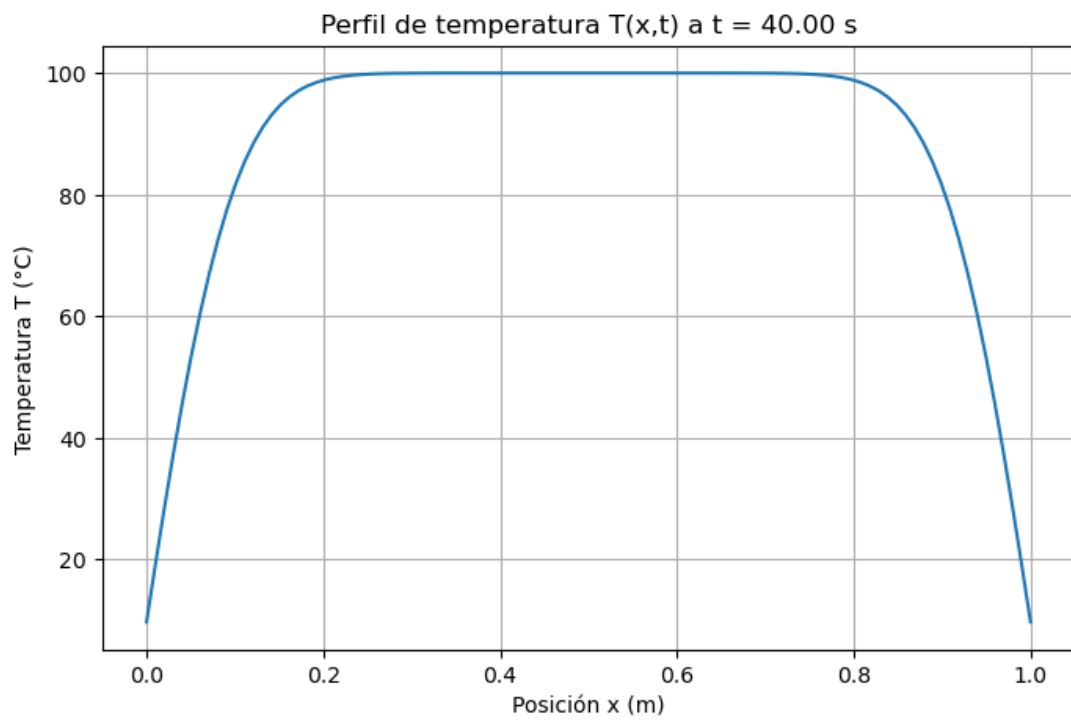


Figura 5: Perfil de temperatura obtenido mediante el código numérico a  $t = 40$  s.

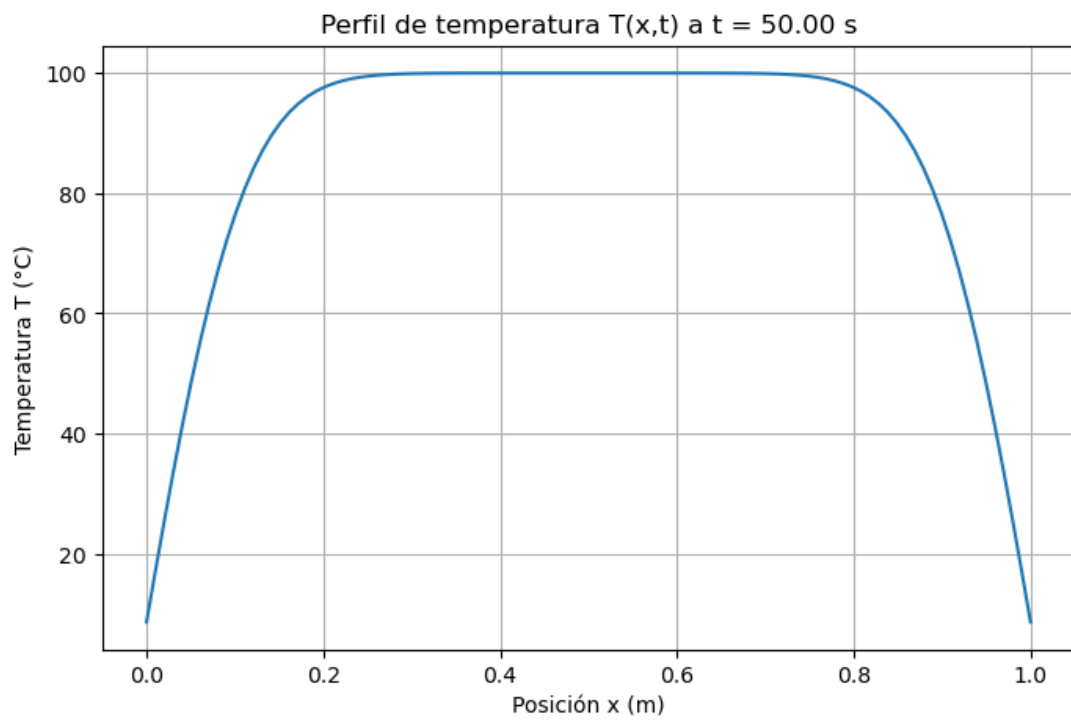


Figura 6: Perfil de temperatura obtenido mediante el código numérico a  $t = 50$  s.

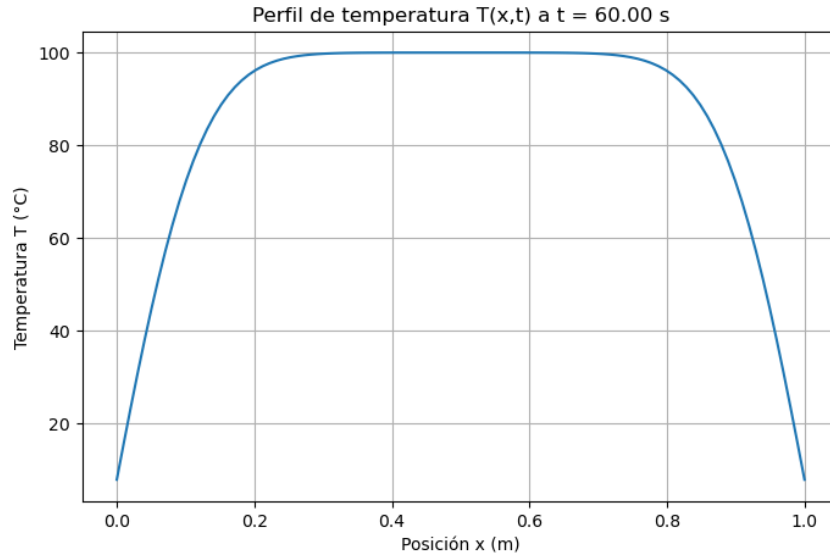


Figura 7: Perfil de temperatura obtenido mediante el código numérico a  $t = 60$  s.

La solución presenta una temperatura cercana a 100 en una región amplia de la barra y una transición suave hacia las fronteras mantenidas en cero. Este comportamiento es consistente con un proceso de difusión: conforme transcurre el tiempo, el calor se desplaza hacia los extremos.

## 10. Comparación: diferencias finitas vs. solución analítica

Para validar el resultado numérico, se superponen el perfil calculado mediante diferencias finitas y el obtenido a partir de la serie de Fourier.

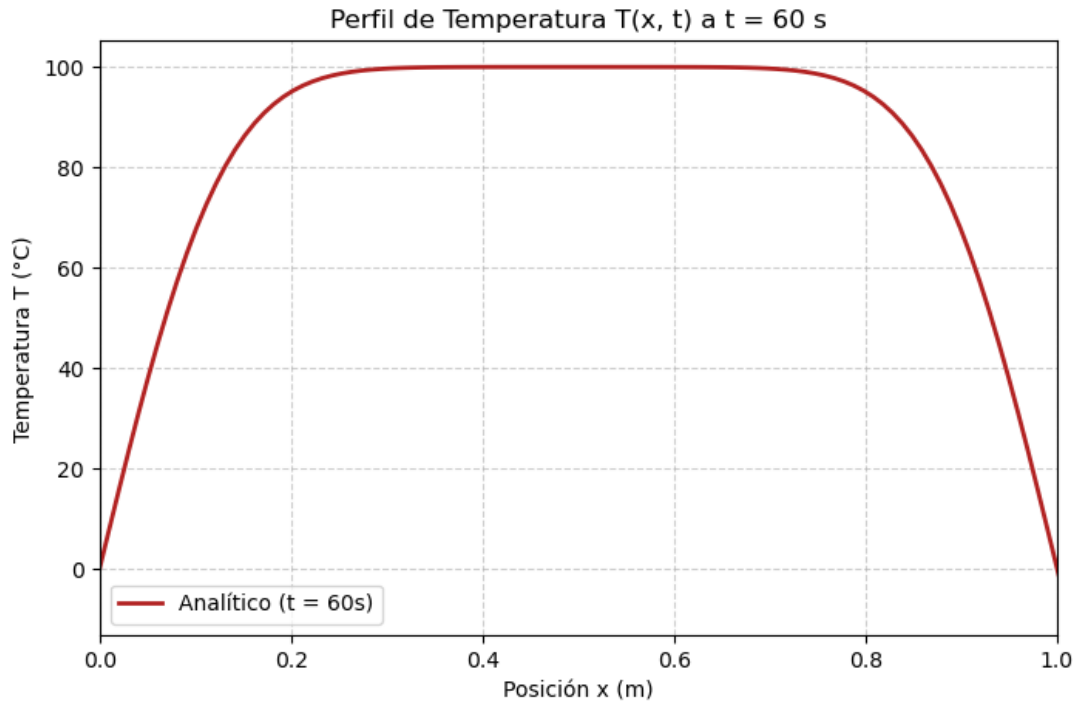


Figura 8: Comparación entre la solución numérica por diferencias finitas y la solución analítica por separación de variables a  $t = 60$  s.

La comparación muestra que ambos enfoques reproducen el mismo comportamiento físico. La solución numérica se aproxima a la solución analítica cuando  $\Delta x$  y  $\Delta t$  son suficientemente pequeños y el esquema se mantiene estable.

## 11. Visualización de todos los estados

La función original para generar los perfiles es

```

1
2 def output(CU, x, dtout): #GRAFICADOR
3     for i, snap in enumerate(CU):
4         plt.figure(figsize=(8, 5))
5         #plt.plot(x, snap)
6         plt.plot(x[1:Nx+1], snap[1:Nx+1])
7         plt.xlabel('x')
8         plt.ylabel('T')
9         plt.title(f'Temperatura (t = {i * dtout:.2f} s)')
10        plt.grid()
11        plt.show()
12
13 dt = timestep()
14 inittemperature(T)
15 boundary(T)
16 T, TP, t, it, CU = step(T, TP, cfl, t, dt, it, tfin, dtout)
17 output(CU, x, dtout)

```

Esta función permite visualizar la evolución temporal de la temperatura. Cada elemento de **CU** representa un estado almacenado durante la simulación.

## 12. Conclusiones

El proyecto implementa una solución numérica de la ecuación de calor unidimensional mediante un esquema explícito de diferencias finitas. La obtención del método parte directamente de las aproximaciones de Taylor para las derivadas temporal y espacial.

El parámetro CFL utilizado produce un coeficiente

$$C = 0.01,$$

que satisface la condición clásica de estabilidad del esquema explícito,  $C \leq 1/2$ .

Por otro lado, la solución analítica obtenida mediante separación de variables permite disponer de una referencia independiente. La expansión en serie de Fourier muestra que, debido a la condición inicial constante y a las fronteras a temperatura cero, únicamente aparecen los modos impares.

La comparación de ambas soluciones demuestra la utilidad de combinar métodos numéricos y analíticos: el método de diferencias finitas permite resolver el problema de manera computacional y flexible, mientras que la solución analítica proporciona una referencia para verificar la implementación.